

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..3

CONFIGURATION

 Copy as Markdown

Configuration

Configuration decides what your server listens on, which database and Redis (an in-memory store Serverpod can use for pub/sub and caching) it connects to, and how it behaves in each run mode, so the same code runs locally, in staging, and in production without edits. Serverpod reads configuration from three sources: the `config/<run-mode>.yaml` files, environment variables, and a `ServerpodConfig` Dart object passed to the `Serverpod` constructor. Environment variables override matching keys in the YAML files, and a Dart object replaces both.

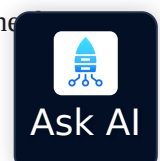
With no configuration at all, Serverpod falls back to built-in defaults, so you can start with nothing and add configuration as you need it.

How configuration works

The YAML files and environment variables combine, and the Dart configuration object replaces both:

- **YAML files** (`config/development.yaml`, `config/staging.yaml`, `config/production.yaml`, `config/test.yaml`): the baseline configuration, one file per run mode.
- **Environment variables**: override matching YAML values, useful for per-deployment settings and secrets.
- **Dart configuration object**: a `ServerpodConfig` passed to the `Serverpod` constructor. When you pass one, the server ignores `config/<run-mode>.yaml` and the `SERVERPOD_*` configuration variables. Command-line options such as `--mode` still override its values.

To adjust the loaded configuration instead of replacing it, pass a `configOverride` function to the `Serverpod` constructor. It receives the configuration from the sources above, and returns the one it uses.



For every available option, its environment variable, config-file key, and default, see the [Configuration reference](#).

Defaults

If no YAML config files exist, no environment variables are configured, and no Dart config is supplied, the server runs with this built-in default: the API server on port 8080 at `localhost`.

```
ServerpodConfig(  
  apiServer: ServerConfig(  
    port: 8080,  
    publicHost: 'localhost',  
    publicPort: 8080,  
    publicScheme: 'http',  
  ),  
);
```

Run modes

The server always runs in one of four run modes: `development` (the default), `test`, `staging`, or `production`. The run mode is selected with the `--mode` argument when the server starts, and it decides what the server loads:

- The matching `config/<run-mode>.yaml` file.
- The matching section of the `passwords file`, merged with its `shared` section.

The `test` mode is used by the `test tools`, which start the server against `config/test.yaml`. The other modes separate your local setup from your deployed environments. See [Running your server](#) for how to pass `--mode` through `serverpod start`.

Run modes configure the server side. Your Flutter app picks the matching server address per environment on its own; see [Point the client at each environment](#).

Configure with YAML files

The `config` directory at the root of the server project holds one file per run mode, created with your project. The server loads the file named after the run mode it starts in: `config/development.yaml` is used when running in the `development` run mode. This is the scaffolded development file for a project named `myproject`, with comments and optional keys removed:

config/development.yaml

```
apiServer:  
  port: 8080
```

```
publicHost: localhost
publicPort: 8080
publicScheme: http

insightsServer:
  port: 8081
  publicHost: localhost
  publicPort: 8081
  publicScheme: http

webServer:
  port: 8082
  publicHost: localhost
  publicPort: 8082
  publicScheme: http

database:
  host: localhost
  port: 8090
  name: myproject
  user: postgres
  dataPath: .serverpod/development/pgdata

redis:
  enabled: false
  host: localhost
  port: 8091

maxRequestSize: 524288

sessionLogs:
  persistentEnabled: true
  consoleEnabled: true
  consoleLogFormat: text
```

The three server blocks configure Serverpod's three servers: the API server your app talks to, the [Insights](#) server used by Serverpod's tooling, and the [web server](#) for serving web content. Redis is optional and ships disabled; see [Redis](#) for how to enable it and connect a client. The `sessionLogs` block controls how session logs are stored and printed; see [Logging](#). Keys not in the scaffolded file cover future calls, websocket ping intervals, and more; see the [Configuration reference](#) for the full list with defaults.

Database backends

Serverpod supports both PostgreSQL and SQLite as database backends.

WARNING

All run modes must use the same database backend: creating migrations fails otherwise. This also keeps your development environment consistent with production.

PostgreSQL

New projects run an embedded PostgreSQL in the `development` and `test` run modes. When `dataPath` is set, the server boots and manages a PostgreSQL in that directory before connecting, so there is no separate database to install or start. The embedded database is for local development and testing only; in production, connect to an external PostgreSQL by omitting `dataPath`:

```
database:
  host: localhost
  port: 8090
  name: database_name
  user: postgres
  maxConnectionCount: 10
```

Set the database password in `passwords.yaml` (`database`) or through `SERVERPOD_PASSWORD_database`.

SQLite

```
database:
  filePath: server.db
```

No database password is required when using SQLite. SQLite doesn't support persistent session logs. `persistentEnabled` still defaults to `true`, so the server warns at startup until you set it to `false`. Console logging is on by default only in `development`. To see session logs in other run modes, set `consoleEnabled`. See [Configure logging](#).

Configure in Dart

To configure Serverpod in Dart, pass an instance of the `ServerpodConfig` class to the `Serverpod` constructor, normally inside the `run` function in your `server.dart` file. When you configure this way, `apiServer` is the one field you must provide.

```
Serverpod(
  args,
  config: ServerpodConfig(
    apiServer: ServerConfig(
      port: 8080,
      publicHost: 'localhost',
      publicPort: 8080,
      publicScheme: 'http',
    ),
    insightsServer: ServerConfig(
      port: 8081,
      publicHost: 'localhost',
      publicPort: 8081,
      publicScheme: 'http',
    ),
  ),
```

```

webServer: ServerConfig(
  port: 8082,
  publicHost: 'localhost',
  publicPort: 8082,
  publicScheme: 'http',
),
),
);

```

Manage secrets

Secrets are declared in the `config/passwords.yaml` file. Serverpod's API reads them with `getPassword`, so this page uses *secret* and *password* interchangeably. The file has a `shared` section, applied in all run modes, and one section per run mode:

config/passwords.yaml

```

shared:
  stripeApiKey: 'sk_test_123...'

development:
  database: 'development_password'
  redis: 'development_password'
  serviceSecret: 'development_service_secret'
  twilioApiKey: 'dev_twilio_key'

production:
  database: 'production_password'
  redis: 'production_password'
  serviceSecret: 'production_service_secret'
  twilioApiKey: 'prod_twilio_key'

```

You can also provide any secret through [environment variables](#).

Built-in secrets

The following table shows the built-in secrets that Serverpod uses for its core functionality. These can be configured either through environment variables or by adding the corresponding key in a run-mode or `shared` section of the passwords file. They are separate from any custom secrets you define.

Environment variable	Passwords file	Default	Description
SERVERPOD_PASSWORD_database	database	-	The password for the database.

Environment variable	Passwords file	Default	Description
SERVERPOD_PASSWORD_serviceSecret	serviceSecret	-	The token used to connect with Insights. Must be at least 20 characters.
SERVERPOD_PASSWORD_redis	redis	-	The password for the Redis server.

Each built-in secret also has a dedicated environment variable: `SERVERPOD_DATABASE_PASSWORD`, `SERVERPOD_SERVICE_SECRET`, and `SERVERPOD_REDIS_PASSWORD`. All environment variables override the passwords file, and the `SERVERPOD_PASSWORD_*` form wins when both are set.

Secrets for first-party packages

For secrets related to first-party Serverpod packages, see their respective documentation:

- **Cloud storage:** see [File uploads](#) for Google Cloud Storage, AWS S3, and Cloudflare R2 secrets, and [Custom cloud storage](#) for local disk.
- **Authentication:** see [Storing Secrets](#) on the Authentication setup page.

Custom secrets

Add your own keys to the passwords file the same way, under `shared` or a run-mode section, as `stripeApiKey` and `twilioApiKey` are in the example above.

Via environment variables

You can also define custom secrets using environment variables with the `SERVERPOD_PASSWORD_` prefix. For example, `SERVERPOD_PASSWORD_myApiKey` becomes available as `myApiKey`: the prefix is stripped. These variables override same-named entries in the passwords file and, like the `shared` section, apply in all run modes.

```
export SERVERPOD_PASSWORD_stripeApiKey=sk_test_123...
```

Access secrets in code

Secrets are only available on the server. They are never sent to or accessible from your Flutter app.

Inside an endpoint, read a secret from the `Session` through the `passwords` map:

```
Future<void> processPayment(Session session, PaymentData data) async {
  final stripeApiKey = session.passwords['stripeApiKey'];
```

```
// Use the API key to make requests to Stripe.  
}
```

The same value is available from `session.serverpod.getPassword('stripeApiKey')`.

Outside of a request, for example during startup in your server's `run` function, read it from the `Serverpod` instance with `getPassword`:

```
// `pod` is the Serverpod instance created in run().  
final stripeApiKey = pod.getPassword('stripeApiKey');
```

This works for built-in and custom secrets alike, whether they come from the passwords file or an environment variable.

Secrets in production

A new project's `.gitignore` excludes `config/passwords.yaml` and credential files such as `config/firebase_service_account_key.json`, so secrets are not committed by default. Keep production secrets out of source control.

In production, set secrets through `SERVERPOD_PASSWORD_*` environment variables, or your host's secret manager, rather than a checked-in passwords file.

Passwords on Serverpod Cloud

On [Serverpod Cloud](#), you set the values that `getPassword` reads from the command line, with `serverpod cloud password set`, instead of editing a passwords file. See [Passwords, secrets, and environment variables](#) for the full reference.

Configure code generation

Serverpod uses a `generator.yaml` file to configure code generation. Place this file in the `config` directory of your server project.

For every `generator.yaml` option, its type and default, see the [Configuration reference](#). The sections below explain the options you set most often.

Package types

The `type` field determines how Serverpod treats your package:

- **server**: A standard Serverpod application (default).
- **module**: A reusable module that can be imported by other Serverpod projects.
- **internal**: Used only by Serverpod's own framework packages; you will not use this.

For modules, you can also specify a `nickname`:

```
type: module
nickname: auth
```

Client package path

By default, Serverpod expects the client package to be located at `../[project_name]_client`. You can customize this:

```
client_package_path: ../my_custom_client
```

Test tools generation

Fresh projects include this line in `generator.yaml`, which generates the integration test tools:

```
server_test_tools_path: test/integration/test_tools
```

Delete the line to stop generating test tools. See the [testing documentation](#) for how the test tools are used.

Module dependencies

Declare module dependencies and optionally assign nicknames for easier reference:

```
modules:
  serverpod_auth_core:
    nickname: auth
  my_custom_module:
    nickname: custom
```

This allows you to reference module classes as `module:auth:AuthUser` in your model files. See the [modules documentation](#) for more information.

Shared packages

Shared packages let you define models that can be imported by both server and client code. They depend only on `serverpod_serialization` and are safe to use across your full stack. Configure them in `generator.yaml`:

```
shared_packages:
  - ../my_shared_package
  - ../another_shared_package
```

Models and the protocol file are generated in each shared package's own directory when you run `serverpod generate` from your server project. See the [shared packages documentation](#) for setup, usage, and restrictions.

Custom serializable classes

Register custom classes for use in your models:

```
extraClasses:  
- package:my_shared_package/my_shared_package.dart:CustomClass  
- package:my_shared_package/my_shared_package.dart:AnotherCustomClass
```

See the [serialization documentation](#) for implementing custom serializable classes.

Features

Control which Serverpod features are enabled:

```
features:  
  database: false
```

A server that does not use a database can set `database: false` to skip migrations and table generation. Projects created without a database, for example with `--no-database`, don't include this setting, so add it yourself.

Experimental features

Experimental features are off by default and opted into per project with the `experimental_features` key. The current one is `databaseSync`, which enables `database: sync` on table models; `all` enables every available feature:

```
experimental_features:  
  databaseSync: true
```

The `--experimental-features databaseSync` command line flag does the same for a single run.

The `experimental_features` key only affects code generation. Experimental runtime APIs are on `pod.experimental` instead: `shutdownTasks` to [run code on shutdown](#), and `submitDiagnosticEvent` for [exception monitoring](#).

WARNING

Experimental features may change or be removed in future versions.

Related

- [Configuration reference](#): every option with its environment variable, config key, and default.
- [Running your server](#): how the server starts and how `--mode` is passed.
- [Database connection](#): connecting to your database in depth.
- [Redis](#): enabling Redis, the password, and connecting a GUI.
- [Configure HTTP calls](#): the API server's response headers and CORS defaults.